

Infra

Morning Flower and Evening Oath

比赛相关、工具软件、语言基础

目录

第一章 比赛相关	1
1.1 提交前的接口核对	1
1.2 对拍与性能检查	1
第二章 工具软件	2
2.1 编译与运行	2
2.2 Bash 文件与退出状态	2
2.3 Python 参考程序	2
第三章 语言基础	3
3.1 值、引用与调用边界	3
3.2 <code>std::move</code> : 允许移动, 不是立即移动	3
3.3 <code>vector</code> 、 <code>array</code> 、 <code>deque</code> 与字符串	3
3.4 <code>set</code> 、 <code>map</code> 与哈希表	3
3.5 <code>queue</code> 、 <code>stack</code> 与优先队列	4
3.6 排序、查找与区间算法	4
3.7 <code>pair</code> 、 <code>tuple</code> 、 <code>optional</code> 与回调	4
3.8 数值、位操作和随机数	4
3.9 GNU 扩展与查询范围	4
第四章 分册跳转索引	5

第一章 比赛相关

本册是当前模板所用标准库与环境内容的第一版整理，不宣称覆盖整个 C++ 标准库。尚需按实际使用记录补齐未登记的标准库细项、环境版本差异与更多经过执行的示例。

1.1 提交前的接口核对

先对照题面确认下标、闭区间或半开区间、空输入、重边、自环和数值范围。返回值为 0 可能是合法答案，不能一律当作失败。算法模板的数学前提与输出方案约定优先于类型名的直觉。

本库 C++20 模板通常需要 `bits/stdc++.h`、`cassert` 和 `std` 命名空间。GNU 扩展不是标准 C++；若评测环境没有 `libstdc++`，不要假定 `PBDS`、`rope`、`bits/stdc++.h` 可用。整数乘积先转宽类型再乘，不能乘完才强转。断言用于检查调用前提，不能把断言本身当作正确性证明。

1.2 对拍与性能检查

小数据参考应尽量用枚举、直接模拟或不同数学公式。固定随机种子并保存失败输入；先检查样例、空边界、极值、重复值，再测试最大规模。消毒器检查越界与未定义行为，优化版单独测时间。相同机器上的本地耗时也不能直接换算为 OJ 排名。

递归 DFS 保持递归。本机默认线程栈较小时调整本机测试环境，不因此改成显式模拟 DFS。算法的栈深度与题目内存约束仍需匹配。

第二章 工具软件

2.1 编译与运行

以下在 Bash 中运行；源文件和输出文件变量加引号。macOS 的 g++ 可能实际调用 Clang，先用 `-version` 核对；本项目在本机选择 g++-16，其他环境可设置 CXX。调试与提交使用相同语言标准，GNU 扩展用 GNU 工具链实测。

```
1 src="main.cpp"
2 out="main"
3 g++ -std=c++20 -O2 -Wall -Wextra "$src" -o "$out"
4 ./main < input.txt > actual.txt
5 diff -u expected.txt actual.txt
```

调试构建：

```
1 g++ -std=c++20 -O1 -g -fsanitize=address,undefined \
2     -fno-omit-frame-pointer main.cpp -o debug
3 ./debug < input.txt
```

Linux 本地测试可用 `ulimit -s unlimited` 放宽允许范围内的主线程栈；macOS 使用 GNU C++ 时本库深递归测试采用链接选项 `-Wl,-stack_size,0x20000000`。这些是本地测试配置，不应盲目加进 OJ 源文件。

2.2 Bash 文件与退出状态

< 重定向输入，> 覆盖输出文件，>> 追加。不要把输入文件名同时作为输出文件名。脚本中可用 `set -euo pipefail` 尽早发现命令失败、未定义变量和管道失败。`diff` 返回 1 通常表示内容不同，不是程序崩溃。

含空格的路径使用双引号；想保留美元符号、反引号等字面内容时，用单引号或带引号的 here-document 结束标记。命令替换不是字符串转义工具，不拼接不可信文本形成命令。

2.3 Python 参考程序

Python 的 `int` 为任意精度整数；`//` 是向下取整，负数时与 C++ 整数除法不同。`pow(a,e,m)` 做模幂；Python 3.8 起，`pow(a,-1,m)` 在可逆时求逆，不可逆会抛异常。`math.isqrt` 返回整数平方根，避免大整数浮点舍入。

```
1 import math
2 import random
3 rng = random.Random(20260913)
4 a = rng.randrange(1, 100)
5 assert (-3) // 2 == -2
6 assert pow(3, -1, 7) == 5
7 assert math.isqrt(10**30) == 10**15
```

大量输入可用 `sys.stdin.buffer`，但一次 `read().split()` 会同时保存全部 token；大样例应估计 Python 对象开销。列表的 `b=a` 是同一个对象，`a[:]` 只复制一层；二维数组用列表推导构造，避免多行共享同一列表。

第三章 语言基础

3.1 值、引用与调用边界

参数 `T` 按值, `const T&` 只读借用, `T&` 允许修改原对象。返回引用必须保证被引用对象仍存活, 不能返回局部变量的引用。结构化绑定 `auto [x,y]` 默认复制, `auto& [x,y]` 引用原元素; 只读遍历常用 `const auto&`。

3.2 `std::move`: 允许移动, 不是立即移动

`std::move(x)` 本身是转换表达式, 使后续重载选择可以使用移动构造或移动赋值; 它自己不搬运数据。对 `const` 对象使用 `move` 不会去掉 `const`, 常见容器因此仍走复制。除有更强保证的特例外, 标准库对象移动后有效但状态未指定: 可销毁、重新赋值, 或调用满足其前提的操作, 不假定内容保持原样或必定为空。

```
1 vector<int> a{1, 2, 3};
2 vector<int> b = std::move(a);
3 a.clear();
4 a.push_back(7);
5 const vector<int> c{4, 5};
6 vector<int> d = std::move(c);
```

上例 `b` 得到原序列, `d` 从 `const c` 复制。调用接受值参数的卷积函数且之后不再需要输入时, 传 `std::move(a)` 可以避免额外副本; 函数参数为 `const` 引用时不会因此取得所有权。返回本地变量通常直接 `return a;`; 多加 `move` 可能失去具名返回值优化机会。依据: [标准草案 move 定义与移动后状态](#)。

3.3 `vector`、`array`、`deque` 与字符串

`vector` 连续存储, 随机访问常数时间, 尾部追加摊还常数时间。`reserve` 只改容量, 不增加元素; `resize` 才改变 `size`。重分配使原指针、引用和迭代器失效; `erase` 使被删位置及之后失效。不要保存 `vector` 节点引用后再 `push` 新节点; 需要跨扩容引用时保存下标并重新访问。[失效规则来源](#)。

`array` 的长度为编译期常量; 局部基本类型元素需用 `{}` 初始化。`deque` 支持两端操作且可下标访问, 但整体不连续; 修改后不要盲目保留迭代器。`vector<bool>` 的元素可能是代理对象, 不等同于普通 `bool` 引用。

`string` 适合字符序列; `substr(pos,len)` 第二个参数是长度。`find` 未找到返回 `npos`, 不能直接转 `int` 再判断。`getline` 读整行, 与 `cin>>` 混用时处理遗留换行。字符串反复头部删除可能线性搬移; 需要复杂序列操作见分册 `rope` 与序列树。

3.4 `set`、`map` 与哈希表

`set/map` 去重, `multiset/multimap` 允许等价键。比较器必须是严格弱序; 不能用 `less_equal` 假装多重集合, 也不直接用不传递的 `epsilon` 比较做排序或树比较。`map` 的下标访问会插入默认值, 只查询用 `find`。

`multiset` 的 `erase(key)` 会删掉全部等价元素; 只删一个时先 `find`, 再 `erase` 该迭代器。树容器的 `lower_bound` 用成员函数, 通用算法在非随机访问迭代器上可能线性走步。关联容器的键不可原地修改为破坏排序的值。[关联容器要求](#)。

`unordered_map` 的普通查改平均常数时间, 最坏可线性; `rehash` 会使迭代器失效。需要容量控制时看其 `reserve/max_load_factor` 接口, 不把这些名字照搬到 `PBDS`。GNU 的 `gp/cc` 哈希表、选型和范围见本册末的分册索引。

3.5 queue、stack 与优先队列

queue 先入先出，stack 后入先出；取 front/top 和 pop 前检查非空。标准 priority_queue 默认大根堆，pop 不返回值，先复制 top 再 pop。它没有按句柄修改和合并接口。

```
1 priority_queue<int, vector<int>, greater<int>> q;
2 q.push(7);
3 q.push(2);
4 int x = q.top();
5 q.pop();
```

需要修改已入堆的键、删除指定元素或合并时，使用数据结构册的 PBDS 配对堆；不要混用两种 priority_queue 的接口或句柄保证。

3.6 排序、查找与区间算法

算法区间通常为半开区间。sort 要求严格弱序，stable_sort 保留等价元素的原顺序；nth_element 只保证第 k 位及两侧分区，平均线性，不等于完整排序。lower/upper_bound 要求对应有序或分区前提，并使用一致比较规则。算法要求。

```
1 sort(a.begin(), a.end());
2 a.erase(unique(a.begin(), a.end()), a.end());
3 auto it = lower_bound(a.begin(), a.end(), 7);
4 if (it != a.end()) cout << *it << '\n';
```

unique 只合并相邻等价值，返回逻辑终点；它不改容器 size。remove/remove_if 同理，配合 erase；C++20 可用适用容器的 erase/erase_if。reverse、rotate、merge、copy 的目标区间必须有效，不能把未扩容的 vector.begin() 当作足够大的输出区间。

3.7 pair、tuple、optional 与回调

pair/tuple 默认按字典序比较，适合把值与唯一编号组合。optional 区分无解与合法零值，先判断再解引用。std::function 可保存不同可调用对象，但有类型擦除开销；泛型 lambda 或模板回调更容易内联。递归 lambda 捕获的引用不能活得比被引用对象更久。

3.8 数值、位操作和随机数

accumulate 的初值决定累加类型，64 位求和用 0LL。iota 适合连续编号。gcd/lcm 仍有可表示范围前提，最小负数的绝对值应先转宽或转无符号幅值。浮点整数精确范围和 long double 有效位数用 numeric_limits 查询；几何数值条件另看几何册。

C++20 的 popcount/countr_zero 等用于无符号类型；标准 countr_zero(0) 有定义。

GCC 的 __builtin_ctz(0) 不能调用。1ULL<<k 仍须保证 k 小于位宽。bitset 的大小固定，批量位运算不是与位数无关的常数操作。

mt19937/mt19937_64 用固定种子可复现对拍；uniform_int_distribution 的两端都包含。rng()%n 一般有偏差，不把它当作严格均匀抽样。随机算法的种子和期望复杂度约定按对应模板说明执行。

3.9 GNU 扩展与查询范围

PBDS 有序树、gp/cc 哈希表、配对堆以及 __gnu_cxx::rope 在所属知识点详细介绍，本册提供集中页码跳转。rope 不叫 std::rope；复制版本的内存成本和已知失败用法不因它是库容器而消失。

第四章 分册跳转索引

请将本册与其他 PDF 放在同一目录。下表页码是分册印刷页码；链接直接指向条目，部分浏览器内置 PDF 阅读器可能不支持跨文件跳转。

内容	所属分册及页码
PBDS 配对堆：修改、合并与句柄	数据结构，第 8 页
PBDS 有序树：排名与第 k 小	数据结构，第 42 页
GNU rope：序列与版本共享	数据结构，第 40 页
gp/cc 哈希表：整数键映射	数据结构，第 1 页
优先队列的最短路应用	图论，第 31 页
vector 与下标：树状数组	数据结构，第 11 页
扩容与下标：可持久化数组	数据结构，第 67 页
比较次序：后缀数组	字符串，第 8 页
数值精度约定：浮点几何	计算几何，第 1 页
移动大数组：NTT 卷积	数学，第 56 页

算法索引

array, 3

Bash, 2

bitset, 4

deque, 3

function, 4

g++, 2

lower-bound, 4

map, 3

multiset, 3

nth-element, 4

numeric, 4

optional, 4

pair, 4

PBDS, 4

pow, 2

priority-queue, 4

Python, 2

queue, 4

random, 4

reference, 3

rope, 4

Sanitizer, 2

set, 3

sort, 4

stack, 4

std::move, 3

string, 3

tuple, 4

unique, 4

unordered-map, 3

vector, 3

对拍, 1

提交检查, 1